

Slide 1 — Decision-Focused Learning for Kidney Exchange Programs

Good afternoon everyone. My name is Weikang Fu, and today I will introduce my internship research project: **decision-focused learning for kidney exchange programs**.

The main idea of this project is to connect machine learning with combinatorial optimization. In other words, it's the optimization application in the medical scenario.

Transition: I will start with the motivation: why kidney exchange is important.

Slide 2 — Why Kidney Exchange Matters

Kidney transplantation is an important treatment for patients with kidney failure. But compatible living donors are scarce. Even when a patient has a willing donor, that donor may be medically incompatible with the patient.

Kidney exchange can help solve this problem. It pools the patient-donor pairs together, and then selects edges between them to form cycles and chains.

The top figure shows a cycle, where donors are exchanged among pairs. The bottom figure shows a chain, where a non-directed donor starts a sequence of transplants that can help multiple patients.

The key point is that kidney exchange creates more transplant opportunities from the same pool of donors.

Transition: Now I will describe how this becomes a graph optimization problem.

Slide 3 — Kidney Exchange as a Graph Optimization Problem

We can model a kidney exchange pool as a directed graph. The vertices are patient-donor pairs and non-directed donors. The edge means that the donor from one vertex can donate to the patient at another vertex.

Then the decision problem is to select cycles and chains. Cycles use only patient-donor pairs, and chains start from a non-directed donor. In practice, these length limits are policy choices, not just mathematical parameters. For example, Germany mainly allows only two-way exchanges, so the cycle limit is $K = 2$. In Canada and the U.S., three-way cycles are allowed in the setting I discuss here, so $K = 3$. That move from 2 to 3 already creates many more candidate cycles.

The reason is practical. Cycles usually require simultaneous surgeries, so most programs keep them short, often 2 or 3 pairs. Chains are different because they can be non-simultaneous. This is why mature KPD systems, especially in the U.S., can support longer non-directed donor chains.

Transition: The next slide gives a small example of what the optimizer has to consider.

This example shows a small KEP graph.

I do not need to go through every chain here. The main point is that the solver first has to generate many candidate cycles and chains, and then select a feasible subset where no vertex is reused.

Transition: After this example, let's talk about two main difficulty in this project.

Slide 5 — What Makes the Problem Difficult?

There are two difficulties. The first is the optimization difficulty. Cycles and chains create a combinatorial problem. Once the cycle length is bounded and larger than two, the problem becomes super hard to compute.

The second difficulty is prediction. The benefit of a transplant edge is uncertain. For example, it may depend on medical conditions, transplant success probability, or many other factors. Also, a good edge-level prediction does not always lead to the best final exchange plan. I think this is a very interesting point, and I will try to explain it in the later talks.

Transition: To see why prediction matters, we first need to define what the optimizer sees in the problem.

Slide 6 — From Transplant Benefit to Edge Weights

For each possible transplant edge, we need a benefit score or sometimes called edge weight. In the optimizer, all of this information is summarized as edge weights. So the optimizer does not directly see the full medical story. It sees a weighted compatibility graph, and then selects cycles and chains using these weights.

In my experiments, these edge benefits are synthetic labels, not real clinical outcomes. Because the real datasets are not shared in most situations. This is a common scenario in medical machine learning. So I will firstly stick to the synthetic data to show the general framework of my project.

Transition: Firstly, I will introduce the standard approach: Predict Then Optimize.

The standard approach is called predict then optimize. In step one, we train a machine learning model to predict edge weights from features. Usually this is done with a prediction loss function such as MSE.

In step two, we pass these predicted weights into the KEP optimizer, here represented by the Gurobi solver, and obtain the selected exchange plan.

The problem is the mismatch at the bottom: prediction loss is not necessarily the same as decision quality. A model can have low prediction error but still induce a bad exchange plan.

Transition: This mismatch motivates the main research idea.

Slide 8 — Research Idea: Learning for the Final Exchange Plan

The research idea is to train for the final exchange plan, not only for weights accuracy.

The example on the right shows the intuition. If all edge weights are multiplied by five, the numerical predictions are very different, but the selected exchange plan is the same. So exact edge-weight recovery is not always the final goal. The goal is to learn weights that induce better exchange plans.

But we cannot directly train this objective easily, because the solver is discrete and not directly differentiable. So instead, we train the reward model with a decision-focused surrogate loss.

Transition: However, end-to-end training through the optimizer is not straightforward.

There are two main challenges.

First, non-differentiability. The KEP solver returns a discrete exchange plan. Small changes in predicted weights may not change the selected plan at all. So the true decision gap has zero gradients in almost everywhere.

Second, computational cost. End-to-end training calls the KEP solver inside the learning loop. Perturbation-based surrogates require many repeated solver calls, so training becomes much more expensive than standard MSE training.

This is why the experimental design needs to be controlled: same graph data, same solver, but different training objectives.

Transition: The next slide shows how I implemented this idea in the experimental pipeline.

This slide shows the end-to-end pipeline. The input is a KEP graph. A predictor produces edge weights. These weights go into the KEP solver, and the solver returns an exchange plan.

The difference from the standard approach is the training signal. Instead of only using MSE on edge weights, the decision-focused approach uses a surrogate based on the final decision.

There are two practical solutions here. The first is surrogate gradients: we use a decision-focused surrogate loss to turn discrete solver outputs into a usable training signal. In this study, FY perturbation is the main surrogate.

The second is Gurobi model reuse: when possible, we reuse the cycle and chain structure and update only the edge weights or objective. This helps reduce the cost of repeated solver calls.

Transition: I firstly checked whether solver reuse can actually help lower the cost of computation.

This slide is an just an engineering check to show that since solver-in-the-loop training is expensive, reusing the model structure helps.

The result is that reuse is faster in every formulation. The gains are especially large when model building dominates the runtime.

So this can really help us to build a benchmark so that if we want to do something about the cost here, this can be a very good starting point.

Transition: Before showing the diagnostic plots, I first define the controlled Stage1 setting.

Before the contour plots, this slide defines the controlled experimental setting. The purpose is to keep the learning problem simple enough that we can visualize what the training objectives are doing.

The reward model is a two-feature linear probe:

$$\hat{w}_e(\theta) = \theta_1 u_e + \theta_2 c_e.$$

Here u_e is the edge utility feature, and c_e is the recipient cPRA feature. So θ_1 controls the utility weight, and θ_2 controls the cPRA weight.

The synthetic labels are generated from a noisy linear rule:

$$w_e^{\text{syn}} = \max\{0, (10u_e + 5c_e)(1 + \xi_e)\}.$$

The coefficient $[10, 5]$ is the noise-free signal, and the multiplicative edge noise has scale $\sigma = 0.10$. These are synthetic edge benefits, not clinical outcomes.

Transition: With this setting fixed, we can now look at how MSE and FY move in the decision landscape.

This experiment is a controlled diagnostic. I use a simple two-feature linear probe, based on utility and recipient cPRA, so that we can visualize the learning landscape.

The contour plots compare MSE training and FY-based decision-focused training. MSE moves toward the clean-label direction, roughly $[10, 5]$, because it tries to fit the synthetic edge labels.

FY behaves differently. It follows a decision-induced path, because its training signal comes from the exchange plans selected by the optimizer, not only from edge-level prediction error.

The key message is that decision-focused training changes the geometry of learning.

Transition: The next slide looks at the same diagnostic from a 3D trajectory view.

The 3D plots separate three things: movement in parameter space, change in decision gap, and change in the FY objective.

One important observation is that the FY objective and the decision gap do not always improve monotonically together. Also, the best decision-gap checkpoint can occur before the final epoch.

This matters because it suggests that checkpoint selection and validation strategy are important. We should not assume that the final epoch is always the best final decision model.

Transition: After this in-sample diagnostic, I tested generalization on held-out and unseen graphs.

This is the main generalization result. The metric is normalized decision gap, and lower is better.

There are three evaluation settings. The first is the original held-out noisy-linear test set with 400 graphs. In this setting, the FY-loss-selected end-to-end model improves over the two-stage baseline.

The second setting is a newly generated unseen noisy-linear test set with 1000 graphs. Here, the advantage becomes weaker and less stable.

The third setting is a realistic-synthetic stress test with 2000 graphs. This uses a different label regime, and under this shift, the two-stage baseline is tied or better.

So the result is nuanced. Decision-focused surrogate training can help in a controlled setting, but the gain is not automatically robust. It depends on validation, label regime, and how well the train and test distributions match.

Transition: I also ran an additional full-pipeline experiment with MLP and GNN predictors.

This slide shows an additional experiment using larger MLP and GNN predictors. I compared two-stage MLP and GNN, end-to-end MLP and GNN, and an oracle that uses ground-truth weights.

The main pattern is consistent with the previous message. End-to-end GNN gives modest and repeatable gains over the two-stage GNN baseline. But end-to-end MLP does not consistently improve over two-stage MLP. Also, the oracle gap remains substantial.

So the benefit is architecture-dependent. End-to-end learning can help, but it is not a guaranteed improvement for every model.

Transition: I will finish with the main takeaways, limitations, and next steps.

Slide 17 — Takeaways: Promising, but Not Yet Robust

The main takeaway is that decision-focused surrogate learning is promising, but not yet robust.

What worked is that DFL changes the learned reward geometry. It can reduce downstream decision gap in controlled settings, and it trains toward final exchange plans rather than only edge prediction.

The main limitations are that the experiments use synthetic labels, not clinical outcomes. Step1 uses a simple two-feature probe. I also used limited seeds and distribution settings, and solver-in-the-loop training is computationally expensive.

For future work, I would like to use richer clinical edge-benefit models, stronger graph predictors, larger unseen tests with multiple seeds, and better validation strategies for surrogate training.

So the final message is: decision-focused surrogate learning gives a useful signal in controlled KEP benchmarks, but robustness remains the key challenge. Thank you, and I am happy to take questions.